WIKIPEDIA
The Free Encyclopedia

Knuth–Morris–Pratt algorithm

In computer science, the **Knuth–Morris–Pratt algorithm** (or **KMP algorithm**) is a string-searching algorithm that searches for occurrences of a "word" `W` within a main "text string" `S` by employing the observation that when a mismatch occurs, the word itself embodies sufficient information to determine where the next match could begin, thus bypassing re-examination of previously matched characters.

The algorithm was conceived by James H. Morris and independently discovered by Donald Knuth "a few weeks later" from automata theory.[1][2] Morris and Vaughan Pratt published a technical report in 1970.[3] The three also published the algorithm jointly in 1977.[1] Independently, in 1969, Matiyasevich[4][5] discovered a similar algorithm, coded by a two-dimensional Turing machine, while studying a string-pattern-matching recognition problem over a binary alphabet. This was the first linear-time algorithm for string matching.[6]

Knuth–Morris–Pratt algorithm

Class	String search
Data structure	String
Worst-case performance	$\Theta(m)$ preprocessing + $\Theta(n)$ matching[note 1]
Worst-case space complexity	$\Theta(m)$

Background

A string-matching algorithm wants to find the starting index `m` in string `S[]` that matches the search word `W[]`.

The most straightforward algorithm, known as the "brute-force" or "naive" algorithm, is to look for a word match at each index `m`, i.e. the position in the string being searched that corresponds to the character `S[m]`. At each position `m` the algorithm first checks for equality of the first character in the word being searched, i.e. `S[m] =? W[0]`. If a match is found, the algorithm tests the other characters in the word being searched by checking successive values of the word position index, `i`. The algorithm retrieves the character `W[i]` in the word being searched and checks for equality of the expression `S[m+i] =? W[i]`. If all successive characters match in `W` at position `m`, then a match is found at that position in the search string. If the index `m` reaches the end of the string then there is no match, in which case the search is said to "fail".

Usually, the trial check will quickly reject the trial match. If the strings are uniformly distributed random letters, then the chance that characters match is 1 in 26. In most cases, the trial check will reject the match at the initial letter. The chance that the first two letters will match is 1 in 26 (1 in 26^2 chances of a match over 26 possible letters). So if the characters are random, then the expected complexity of searching string `S[]` of length n is on the order of n comparisons or $O(n)$. The expected performance is very good. If `S[]` is 1 million characters and `W[]` is 1000 characters, then the string search should complete after about 1.04 million character comparisons.

That expected performance is not guaranteed. If the strings are not random, then checking a trial `m` may take many character comparisons. The worst case is if the two strings match in all but the last letter. Imagine that the string `S[]` consists of 1 million characters that are all A, and that the word `W[]` is 999 A characters terminating in a final B character. The simple string-matching algorithm will now examine 1000 characters at each trial position before rejecting the match and advancing the trial position. The simple string search example would now take about 1000 character comparisons times 1 million positions for 1 billion character comparisons. If the length of `W[]` is k, then the worst-case performance is $O(k \cdot n)$.

The KMP algorithm has a better worst-case performance than the straightforward algorithm. KMP spends a little time precomputing a table (on the order of the size of `W[]`, $O(k)$), and then it uses that table to do an efficient search of the string in $O(n)$.

The difference is that KMP makes use of previous match information that the straightforward algorithm does not. In the example above, when KMP sees a trial match fail on the 1000th character (`i = 999`) because `S[m+999]` ≠ `W[999]`, it will increment `m` by 1, but it will know that the first 998 characters at the new position already match. KMP matched 999 A characters before discovering a mismatch at the 1000th character (position 999). Advancing the trial match position `m` by one throws away the first A, so KMP knows there are 998 A characters that match `W[]` and does

not retest them; that is, KMP sets `i` to 998. KMP maintains its knowledge in the precomputed table and two state variables. When KMP discovers a mismatch, the table determines how much KMP will increase (variable `m`) and where it will resume testing (variable `i`).

KMP algorithm

Example of the search algorithm

To illustrate the algorithm's details, consider a (relatively artificial) run of the algorithm, where `W` = "ABCDABD" and `S` = "ABC ABCDAB ABCDABCDABDE". At any given time, the algorithm is in a state determined by two integers:

- `m`, denoting the position within `S` where the prospective match for `W` begins,
- `i`, denoting the index of the currently considered character in `W`.

In each step the algorithm compares `S[m+i]` with `W[i]` and increments `i` if they are equal. This is depicted, at the start of the run, like

```
            1         2
m: 012345678901234567890123
S: ABC ABCDAB ABCDABCDABDE
W: ABCDABD
i: 0123456
```

The algorithm compares successive characters of `W` to "parallel" characters of `S`, moving from one to the next by incrementing `i` if they match. However, in the fourth step `S[3] = ' '` does not match `W[3] = 'D'`. Rather than beginning to search again at `S[1]`, we note that no `'A'` occurs between positions `1` and `2` in `S`; hence, having checked all those characters previously (and knowing they matched the corresponding characters in `W`), there is no chance of finding the beginning of a match. Therefore, the algorithm sets `m = 3` and `i = 0`.

```
            1         2
m: 012345678901234567890123
S: ABC ABCDAB ABCDABCDABDE
W:    ABCDABD
i:    0123456
```

This match fails at the initial character, so the algorithm sets `m = 4` and `i = 0`

```
            1         2
m: 012345678901234567890123
S: ABC ABCDAB ABCDABCDABDE
W:     ABCDABD
i:     0123456
```

Here, `i` increments through a nearly complete match `"ABCDAB"` until `i = 6` giving a mismatch at `W[6]` and `S[10]`. However, just prior to the end of the current partial match, there was that substring `"AB"` that could be the beginning of a new match, so the algorithm must take this into consideration. As these characters match the two characters prior to the current position, those characters need not be checked again; the algorithm sets `m = 8` (the start of the initial prefix) and `i = 2` (signaling the first two characters match) and continues matching. Thus the algorithm not only omits previously matched characters of `S` (the `"AB"`), but also previously matched characters of `W` (the prefix `"AB"`).

```
            1         2
m: 012345678901234567890123
S: ABC ABCDAB ABCDABCDABDE
W:         ABCDABD
i:         0123456
```

This search at the new position fails immediately because `W[2]` (a `'C'`) does not match `S[10]` (a `' '`). As in the first trial, the mismatch causes the algorithm to return to the beginning of `W` and begins searching at the mismatched character position of `S`: `m = 10`, reset `i = 0`.

```
            1         2
m: 012345678901234567890123
S: ABC ABCDAB ABCDABCDABDE
W:           ABCDABD
i:           0123456
```

The match at `m=10` fails immediately, so the algorithm next tries `m = 11` and `i = 0`.

```
            1         2
m: 01234567890123456789012
S: ABC ABCDAB ABCDABCDABDE
W:            ABCDABD
i:            0123456
```

Once again, the algorithm matches `"ABCDAB"`, but the next character, `'C'`, does not match the final character `'D'` of the word `W`. Reasoning as before, the algorithm sets `m = 15`, to start at the two-character string `"AB"` leading up to the current position, set `i = 2`, and continue matching from the current position.

```
            1         2
m: 01234567890123456789012
S: ABC ABCDAB ABCDABCDABDE
W:                ABCDABD
i:                0123456
```

This time the match is complete, and the first character of the match is `S[15]`.

Description of pseudocode for the search algorithm

The above example contains all the elements of the algorithm. For the moment, we assume the existence of a "partial match" table `T`, described below, which indicates where we need to look for the start of a new match when a mismatch is found. The entries of `T` are constructed so that if we have a match starting at `S[m]` that fails when comparing `S[m + i]` to `W[i]`, then the next possible match will start at index `m + i - T[i]` in `S` (that is, `T[i]` is the amount of "backtracking" we need to do after a mismatch). This has two implications: first, `T[0] = -1`, which indicates that if `W[0]` is a mismatch, we cannot backtrack and must simply check the next character; and second, although the next possible match will *begin* at index `m + i - T[i]`, as in the example above, we need not actually check any of the `T[i]` characters after that, so that we continue searching from `W[T[i]]`. The following is a sample pseudocode implementation of the KMP search algorithm.

```
algorithm kmp_search:
    input:
        an array of characters, S (the text to be searched)
        an array of characters, W (the word sought)
    output:
        an array of integers, P (positions in S at which W is found)
        an integer, nP (number of positions)

    define variables:
        an integer, j ← 0 (the position of the current character in S)
        an integer, k ← 0 (the position of the current character in W)
        an array of integers, T (the table, computed elsewhere)

    let nP ← 0

    while j < length(S) do
        if W[k] = S[j] then
            let j ← j + 1
            let k ← k + 1
            if k = length(W) then
                (occurrence found, if only first occurrence is needed, m ← j - k  may be returned here)
                let P[nP] ← j - k, nP ← nP + 1
                let k ← T[k] (T[length(W)] can't be -1)
        else
            let k ← T[k]
            if k < 0 then
                let j ← j + 1
                let k ← k + 1
```

Efficiency of the search algorithm

Assuming the prior existence of the table `T`, the search portion of the Knuth–Morris–Pratt algorithm has complexity $O(n)$, where n is the length of `S` and the O is big-O notation. Except for the fixed overhead incurred in entering and exiting the function, all the computations are performed in the **`while`** loop. To bound the number of iterations of this loop; observe that `T` is constructed so that if a match which had begun at `S[m]` fails while comparing `S[m + i]` to `W[i]`, then the next possible match must begin at `S[m + (i - T[i])]`. In particular, the next possible match must occur at a higher index than `m`, so that `T[i] < i`.